Apache Airflow - Coding Assessment

Apache Airflow:

Apache Airflow is an **open-source workflow orchestration platform** used to **author, schedule, and monitor data pipelines**.
It lets you write workflows as Python code (DAGs – Directed Acyclic Graphs) and provides a web UI to visualize, manage, and troubleshoot them.

In simple terms → Airflow is like a "scheduler + pipeline manager" that helps automate tasks and ensures they run in the right order at the right time.

Features of Apache Airflow

1. Dynamic

- Pipelines are defined in Python, so you can use loops, conditionals, configs, and parameters to **generate dynamic workflows**.
- Example: Create 100 tasks in a DAG with just a loop, instead of manually writing 100 tasks.

2. Extensible

- You can create **custom operators, hooks, sensors**, and plugins.
- Thousands of ready-made **providers** exist (Postgres, MySQL, AWS, GCP, Azure, Databricks, Spark, etc.).
- Makes it easy to adapt to any system.

3. Elegant

- DAGs are **explicit and readable** → easy to understand dependencies.
- UI provides **Graph View**, **Grid View**, **Code View** → so you can see structure, execution history, and source code clearly.

4. Scalable

- Uses a **modular architecture** (Scheduler, Webserver, Workers, Metadata DB).
- Supports **Celery / Kubernetes Executors** to distribute tasks across many machines.
- Can handle very large, complex workflows.

5. Monitoring & Visibility

- **Web UI** shows real-time task status (success, failure, retries).
- **Logs available per task** for easy debugging.
- Supports **alerts & notifications** (email, Slack, etc.).

6. Scheduling

- Cron-style or custom schedules (@daily, 0 0 * * *, etc.).
- Catchup for past runs, and backfilling missed runs.
- Can also run tasks **manually** on demand.

7. Integration Friendly

- Works with **databases, cloud services, big data tools** (Hadoop, Spark, Snowflake, etc.).
- APIs and CLI to trigger DAGs programmatically.

8. Community & Open Source

- Large community → continuous improvements and providers.
- Backed by Apache Software Foundation.

Building A Pipeline in Apache Airflow:

Step 1: Initial Setup:

To run our pipeline, we need a working Airflow environment. Docker Compose makes this easy and safe – no system-wide installs required. Just open your terminal and run the following:

```
# Download the docker-compose.yaml file
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'
# Make expected directories and set an expected environment.
mkdir -p ./dags ./logs ./plugins
# In the same directory create a .env file and save this code:
AIRFLOW_IMAGE_NAME=apache/airflow:2.4.2
AIRFLOW_UID=50000
# Initialize the database
docker compose up airflow-init
# Start up all services
docker compose up
```

Once Airflow is up and running, visit the UI at http://localhost:8080.

Log in with: Username: admin
Password: admin

You'll land in the Airflow dashboard, where you can trigger DAGs, explore logs, and manage your environment.

Step 2: Create a Postgres connection

Before our pipeline can write to Postgres, we need to tell Airflow how to connect to it. In the UI, open the Admin > Connections page and click the + button to add a new connection.

Fill in the following details:

Connection ID: tutorial_pg_conn

Connection Type: postgres

Host: postgres

Database: airflow (this is the default database)

Login: airflow

Password: airflow

Port: 5432

Save the connection. This tells Airflow how to reach the Postgres database running in your Docker environment.

Next, we'll start building the pipeline that uses this connection.

Step 3: Create tables for staging and final data

Let's begin with table creation. We'll create two tables:

employees_temp: a staging table used for raw data

employees: the cleaned and deduplicated destination

We'll use the SQLExecuteQueryOperator to run the SQL statements needed to create these tables.

```
        from airflow.providers.common.sql.operators.sql import
SQLExecuteQueryOperator
        create_employees_table = SQLExecuteQueryOperator(
            task_id="create_employees_table",
            conn_id="tutorial_pg_conn",
            sql="""
                CREATE TABLE IF NOT EXISTS employees (
                    "Serial Number" NUMERIC PRIMARY KEY,
                    "Company Name" TEXT,
                    "Employee Markme" TEXT,
                    "Description" TEXT,
                    "Leave" INTEGER
                );""",
        )
        create_employees_temp_table = SQLExecuteQueryOperator(
            task_id="create_employees_temp_table",
            conn_id="tutorial_pg_conn",
            sql="""
                DROP TABLE IF EXISTS employees_temp;
                CREATE TABLE employees_temp (
                    "Serial Number" NUMERIC PRIMARY KEY,
                    "Company Name" TEXT,
                    "Employee Markme" TEXT,
                    "Description" TEXT,
                    "Leave" INTEGER
                );""",
        )
```

Step 4: Load data into the staging table

Next, we'll download a CSV file, save it locally, and load it into employees_temp using the PostgresHook.

```
import os
import requests
from airflow.sdk import task
from airflow.providers.postgres.hooks.postgres import PostgresHook


@task
def get_data():
    # NOTE: configure this as appropriate for your airflow environment
    data_path = "/opt/airflow/dags/files/employees.csv"
    os.makedirs(os.path.dirname(data_path), exist_ok=True)

    url = "https://raw.githubusercontent.com/apache/airflow/main/airflow-core/docs/tutorial/pipeline_example.csv"

    response = requests.request("GET", url)
    with open(data_path, "w") as file:
        file.write(response.text)
    postgres_hook = PostgresHook(postgres_conn_id="tutorial_pg_conn")
    conn = postgres_hook.get_conn()
    cur = conn.cursor()
    with open(data_path, "r") as file:
        cur.copy_expert(
```

```
            "COPY employees_temp FROM STDIN WITH CSV HEADER
DELIMITER AS ',' QUOTE '\"'",
            file,
        )
    conn.commit()
```

Step 5: Merge and clean the data

Now let's deduplicate the data and merge it into our final table. We'll write a task that runs a SQL INSERT … ON CONFLICT DO UPDATE.

```
from airflow.sdk import task
from airflow.providers.postgres.hooks.postgres import PostgresHook
@task
def merge_data():
    query = """
        INSERT INTO employees
        SELECT *
        FROM (
            SELECT DISTINCT *
            FROM employees_temp
        ) t
        ON CONFLICT ("Serial Number") DO UPDATE
        SET
            "Employee Markme" = excluded."Employee Markme",
            "Description" = excluded."Description",
            "Leave" = excluded."Leave";"""
    try:
        postgres_hook =
PostgresHook(postgres_conn_id="tutorial_pg_conn")
        conn = postgres_hook.get_conn()
```

```
        cur = conn.cursor()
        cur.execute(query)
        conn.commit()
        return 0
    except Exception as e:
        return 1
```

Step 6: Defining the DAG

Now that we've defined all our tasks, it's time to put them together into a DAG.

```
import datetime
import pendulum
import os
import requests
from airflow.sdk import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.common.sql.operators.sql import
SQLExecuteQueryOperator

@dag(
    dag_id="process_employees",
    schedule="0 0 * * *",
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    dagrun_timeout=datetime.timedelta(minutes=60),
)
def ProcessEmployees():
    create_employees_table = SQLExecuteQueryOperator(
        task_id="create_employees_table",
        conn_id="tutorial_pg_conn",
```

```
    sql="""
        CREATE TABLE IF NOT EXISTS employees (
            "Serial Number" NUMERIC PRIMARY KEY,
            "Company Name" TEXT,
            "Employee Markme" TEXT,
            "Description" TEXT,
            "Leave" INTEGER
        );""",
)
create_employees_temp_table = SQLExecuteQueryOperator(
    task_id="create_employees_temp_table",
    conn_id="tutorial_pg_conn",
    sql="""
        DROP TABLE IF EXISTS employees_temp;
        CREATE TABLE employees_temp (
            "Serial Number" NUMERIC PRIMARY KEY,
            "Company Name" TEXT,
            "Employee Markme" TEXT,
            "Description" TEXT,
            "Leave" INTEGER
        );""",
)

@task
def get_data():
    # NOTE: configure this as appropriate for your airflow environment
    data_path = "/opt/airflow/dags/files/employees.csv"
    os.makedirs(os.path.dirname(data_path), exist_ok=True)
```

```
        url =
"https://raw.githubusercontent.com/apache/airflow/main/airflow-
core/docs/tutorial/pipeline_example.csv"
        response = requests.request("GET", url)
        with open(data_path, "w") as file:
            file.write(response.text)

        postgres_hook =
PostgresHook(postgres_conn_id="tutorial_pg_conn")
        conn = postgres_hook.get_conn()
        cur = conn.cursor()
        with open(data_path, "r") as file:
            cur.copy_expert(
                "COPY employees_temp FROM STDIN WITH CSV
HEADER DELIMITER AS ',' QUOTE '\"'",
                file,
            )
        conn.commit()

    @task
    def merge_data():
        query = """
            INSERT INTO employees
            SELECT *
            FROM (
                SELECT DISTINCT *
                FROM employees_temp
            ) t
            ON CONFLICT ("Serial Number") DO UPDATE
```

```
            SET
              "Employee Markme" = excluded."Employee Markme",
              "Description" = excluded."Description",
              "Leave" = excluded."Leave";
        """
        try:
            postgres_hook = PostgresHook(postgres_conn_id="tutorial_pg_conn")
            conn = postgres_hook.get_conn()
            cur = conn.cursor()
            cur.execute(query)
            conn.commit()
            return 0
        except Exception as e:
            return 1
    [create_employees_table, create_employees_temp_table] >> get_data() >> merge_data()

dag = ProcessEmployees()
```

Save this DAG as dags/process_employees.py. After a short delay, it will show up in the UI.

Step 7: Trigger and explore your DAG

Open the Airflow UI and find the process_employees DAG in the list. Toggle it “on” using the slider, then trigger a run using the play button.

You can watch each task as it runs in the Grid view, and explore logs for each step.x

---

View a Pipeline in Apache Airflow:

Once a DAG (Directed Acyclic Graph) is created and placed inside the dags/ folder, Apache Airflow automatically detects and parses it. The following steps describe how to view and interact with the pipeline using the Airflow web interface:

1. Access the Airflow Web UI:

- Open a browser and navigate to http://localhost:8080 (or the configured host/port).
- Log in with your Airflow credentials.
- The DAGs Home Page will display a list of all available DAGs.

2. Locate the Pipeline

- Use the search bar or scroll through the DAGs list to find your pipeline (by its dag_id).
- Verify that the DAG appears without import errors (check the "Import Errors" link if needed).

3. Enable the DAG

- Ensure the DAG is switched ON (unpaused) using the toggle button next to its name.
- This allows Airflow to schedule it automatically according to its defined schedule.

4. Trigger the Pipeline

- To run the pipeline immediately, click the Trigger DAG button (▶).
- This creates a new DAG Run, independent of the schedule.

5. Monitor Execution in Grid View

- Click on the DAG name to open its details.

- Select the Grid View:
- Columns = individual DAG runs.
- Rows = tasks within the DAG.
- Each square indicates task status (e.g., success, failed, running, skipped).
- Click a task square to view its metadata and execution details.

6. Inspect Logs

- From the task details panel, click Log to open detailed logs.
- Logs show execution flow, SQL queries (if applicable), error traces, and retries.
- This is the primary source for debugging issues.

7. Visualize Dependencies in Graph View

- Switch to the Graph View to see the pipeline structure.
- Nodes represent tasks; edges represent dependencies.
- The Graph view helps validate task sequencing and observe which tasks are pending, running, or completed.

8. Analyze Performance (Optional)

- Gantt Chart: Shows task start/end times and overlaps for a single run.
- Task Duration View: Displays historical task durations across multiple runs to identify performance patterns or bottlenecks.

9. Review the DAG Code

- The Code tab shows the exact Python file that Airflow parsed for the DAG.
- This allows quick verification of the deployed pipeline definition.
